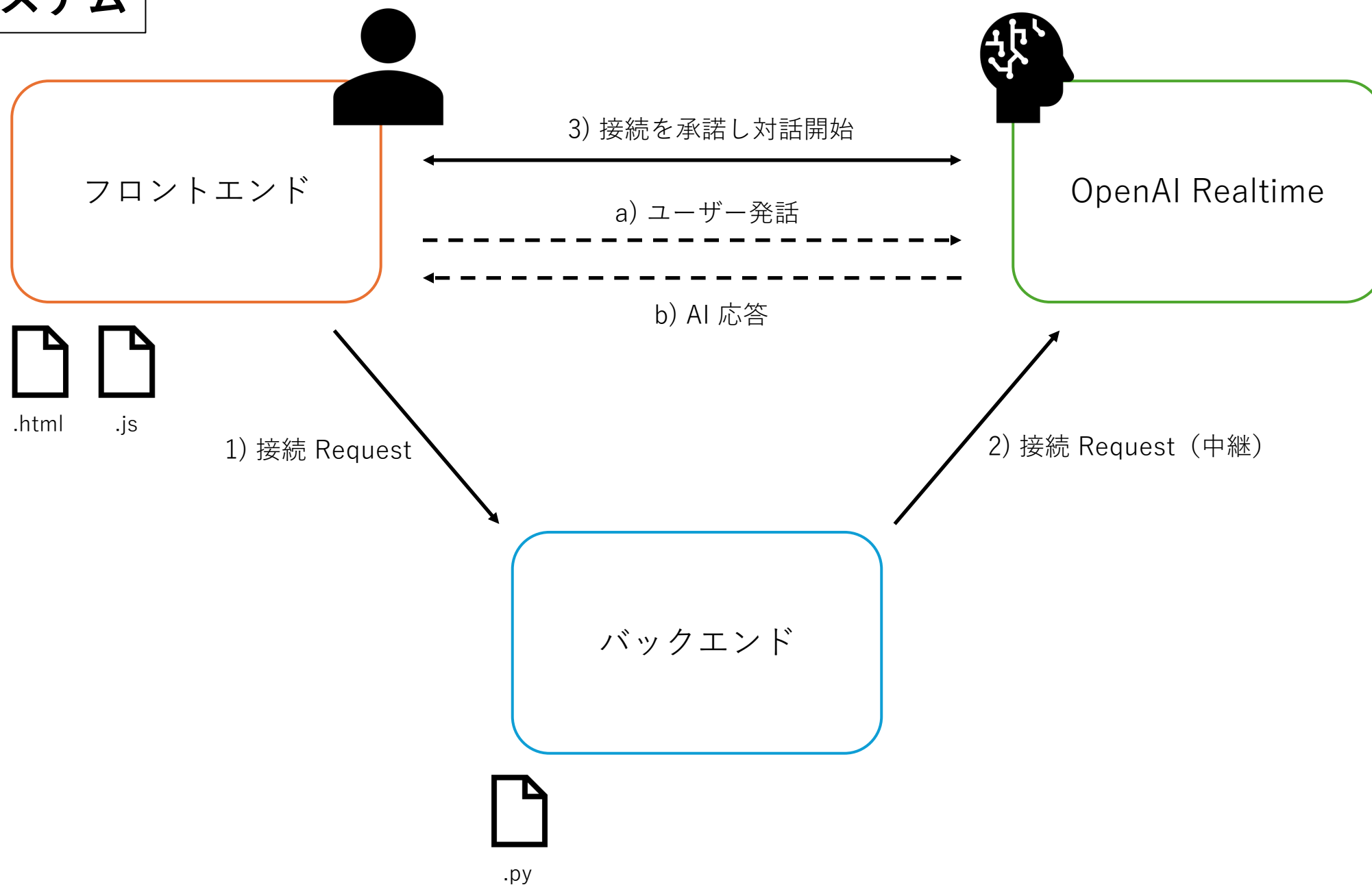


基本システム



フロントエンド

UI 状態管理

画面の見た目とボタン動作を
“状態”ごとに切り替えます

1. agentLoading

「エージェントを準備しています」

2. ready

「おはなしをはじめる 」

3. connecting

「AIに接続しています」

4. active

メインの対話画面（録音中）

5. muted

メインの対話画面（ミュート中）

WebRTC

マイク音声ストリームを取得し
OpenAI と“直接につながる”

1. マイク取得

2. Offer/Answer の交換

通信の決まりを決定する

3. 音声再生開始

Answerを承諾し通信開始

DataChannel

OpenAI と双方向でやり取りし、
UI／アバターを動かす

基本となるイベント

- ① ユーザー発話テキスト化
- ② AI応答テキスト化
- ③ 応答開始（口パク開始）
- ④ 応答終了（口パク終了）

Function Calling関連

- ⑤ 関数呼び出し開始
- ⑥ 関数呼び出しの引数チャンク
- ⑦ 関数呼び出し完了

バックエンド

Request 中継／Ephemeral Key の取得

中継

ブラウザから SDP Offer を受け取り → OpenAI Realtime API に転送 → 返ってきた SDP Answer をクライアントに返却

Ephemeral Key の取得時の主なパラメータ

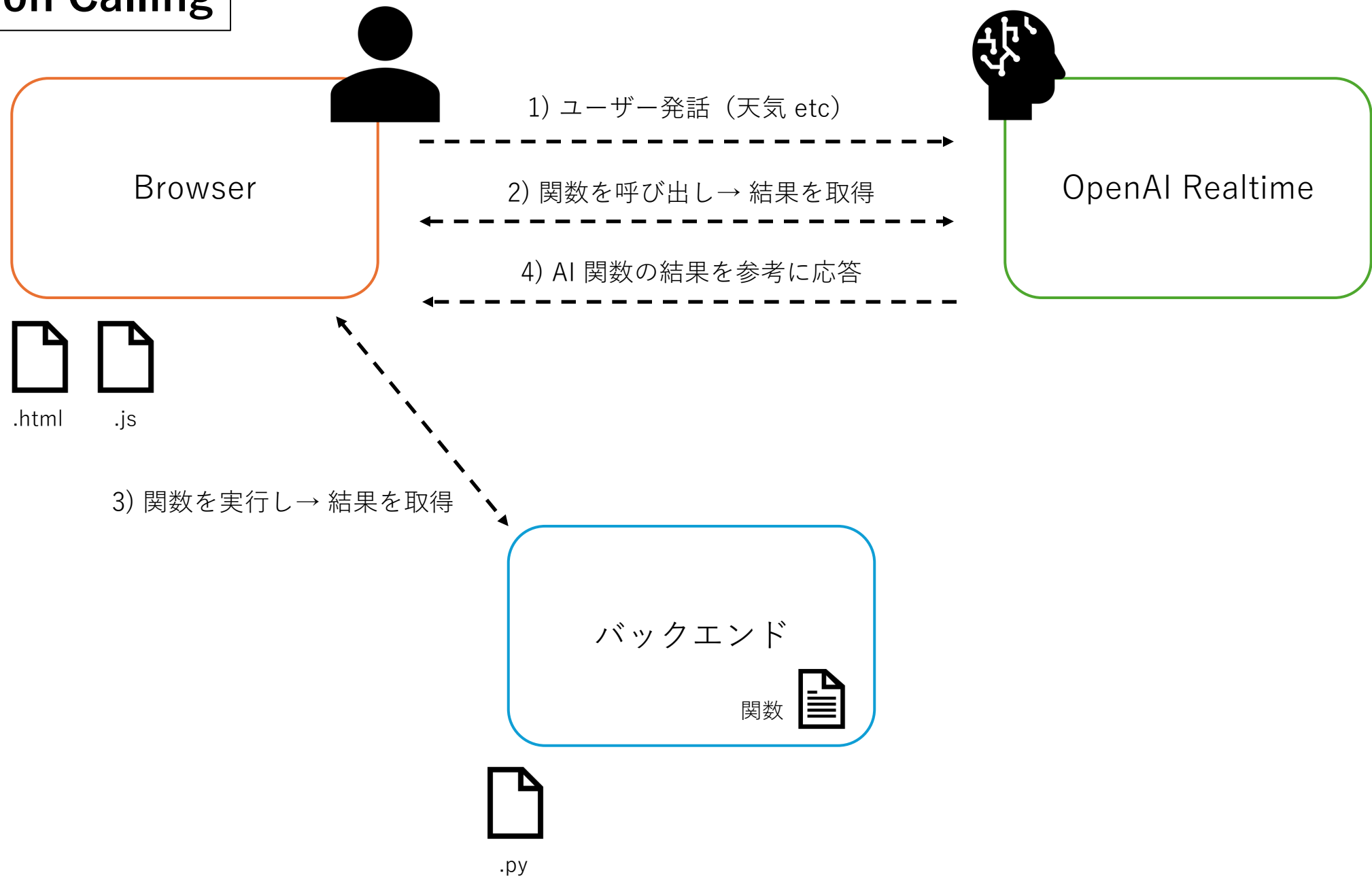
パラメータ	説明
model	利用する会話モデル名
instructions	システムプロンプト（AI の人格・振る舞いを定義）
voice	音声合成の声種（例: shimmer）
input_audio_transcription	音声 → テキスト変換設定
turn_detection	発話検出設定（threshold：検出感度、 silence_duration_ms：無音判定時間）
temperature	応答のランダム性（0.6～1.0）
max_response_output_tokens	最大トークン長

環境構築→実行方法の手順説明

1. Git Hub からファイルをローカルに持ってくる
→ **git clone https://github.com/TeradaLab-Agents/SSH-Realtime-Agent.git**
2. SSH-Realtime-Agent フォルダの中に、5つのファイルを確認
 - ① 対話エージェントシステム
 - A) pharmacy_agent.html
 - B) pharmacy_agent.js
 - C) server_original.py / server_function-calling.py
 - ② 環境構築用ファイル
 - A) requirements.txt
3. python の環境構築
→ SSH-Realtime-Agent で、**pip install -r requirements.txt**
4. アプリの実行
→ SSH-Realtime-Agent で、**uvicorn server_original:app --reload**
uvicorn server_function-calling:app --reload

http://127.0.0.1:8000 となっているか確認
5. pharmacy_agent.html を開く

Function Calling



バックエンド

Request 中継／Ephemeral Key の取得

Ephemeral Key の取得時の主なパラメータ

パラメータ	説明
model	利用する会話モデル名
instructions	システムプロンプト
voice	音声合成の声種（例: shimmer）
input_audio_transcription	音声 → テキスト変換設定
turn_detection	発話検出設定
temperature	応答のランダム性（0.6～1.0）
max_response_output_tokens	最大トークン長
tools	Function Calling 用の関数定義リスト

Function Calling 実行の WebSocket

1. リクエスト受信
2. 関数を実行
3. 関数の実行結果を送信

バックエンド

tools のパラメータ

フィールド	説明
type	この定義全体の型。必ず "object" を指定。
name	関数の識別名。AI が呼び出すときに使うキー。
description	関数が何をするかを AI に伝える説明文。
parameters	引数定義をまとめたオブジェクト。
└─ parameters. type	引数定義全体の型。必ず "object"。
└─ parameters. properties	各引数の詳細をプロパティごとに定義。
└─── parameters.properties. search_query	search_query 引数の定義オブジェクト。
└─── parameters.properties.search_query. type	search_query の型。必ず "string"。
└─── parameters.properties.search_query. description	search_query が何を表すかの説明。
└─ parameters. required	必須の引数名リスト。呼び出し時に必ず渡す必要がある。

フロントエンド

UI 状態管理

WebRTC

DataChannel

Function Calling関連

⑤ 関数呼び出し開始

モデルが「関数を呼び出します」と宣言すると呼び出される
呼び出しID、関数名、引数（最初は空）を取得できる

⑥ 関数呼び出しの引数チャンク

1文字ずつ送られてくるので、文字列に連結する

⑦ 関数呼び出し完了

引数の送信が終わる時に呼び出される
完全版の引数を WebSocket 経由でバックエンドの関数を実行する

用語

API (エーピーアイ)

ソフトウェアやサービス同士が、情報をやりとりするための「つなぎ役」のこと

DataChannel (データチャンネル)

音声や映像のほかに、テキストなどのデータをやりとりするための専用の通り道のこと。

Ephemeral Key (エフェメラル キー)

短い時間だけ使える「使い捨ての合鍵」のようなもの。安全に通信を始めるときに使う。

Function Calling (ファンクション コーリング)

AIが会話の途中で、天気予報を調べるなどの特別な技（関数）を呼び出して使うこと。

Git Hub (ギットハブ)

プログラムの設計図（ソースコード）をインターネット上に保存しておくための場所。

git clone (ギット クローン)

Git Hubにあるプログラムの設計図を、自分のパソコンに丸ごとコピーして持ってくるための命令。

HTML (.html)

ウェブページの文章の構造や骨組みを作るための言語。

JavaScript (.js)

ウェブページにアニメーションなどの「動き」をつけるためのプログラミング言語。

Offer/Answer (オファー/アンサー)

通信を始める前に、お互いに「こんな方法で話そう」とルールの確認をすること。

OpenAI Realtime (オープンエーアイ リアルタイム)

まるで電話のように、AIとリアルタイムでスムーズに音声会話ができる技術のこと。

pip install (ピップ インストール)

プログラムを動かすのに必要な「部品（ライブラリ）」を、パソコンに集める命令。

Python (.py)

今回のシステムの裏側（バックエンド）などで使われている、プログラミング言語。

requirements.txt (リクワイアメンツ テキスト)

プログラムを動かすのに必要な「部品リスト」が書かれているファイル。

SDP (エスディーピー)

WebRTCで通信をするときに交換される「通信のお約束リスト」のこと。

temperature (テンパレチャー)

AIの答えの「ユニークさ」を調整する設定。数値が高いほど、AIは自由で独創的な答えを出しやすい。

UI (ユーアイ)

「ユーザーインターフェース」の略。アプリを使うときに目にする画面や、操作するボタンのこと。

WebRTC (ウェブ アールティーマシー)

ウェブブラウザを通じて、マイクやカメラの音声や映像をリアルタイムにやりとりするための技術。

WebSocket (ウェブソケット)

ウェブサーバーと君のパソコンの間で、いつでもお互いにデータを送りあえるように、

通信ルートをつなぎっぱなしにしておく技術。

システムプロンプト

AIに与える「役割設定」や「性格」に関する台本のようなもの。これでAIの振る舞いが決まる。

トークン

AIが文章を理解したり、作り出したりするときの、いちばん小さな言葉の単位のこと。

バックエンド

ウェブサービスやアプリの「裏側」で動く部分。

ユーザーの目には見えないけれど、データの計算や保存といった大切な仕事をしている。

パラメータ

AIの性格や声の種類、答え方などを細かく調整するための「設定」のこと。

フロントエンド

ウェブサービスやアプリで、みんなが直接見たり触ったりする「表側」の部分。

Function Calling - ② 時刻

最初の import/pip

```
'''② 時刻'''  
import datetime  
from zoneinfo import ZoneInfo
```

3. Function Calling 用ツール定義

```
'''② 時刻'''  
tool_app_rag = {  
    "type": "function",  
    "name": "appRAG",  
    "description": "現在時刻・日付を取得します。（location は無視されます）",  
    "parameters": {  
        "type": "object",  
        "properties": {  
            "location": {  
                "type": "string",  
                "description": "使用しませんが一応受け取ります。"  
            }  
        },  
        "required": []  
    }  
}
```

4. システムプロンプト

```
'''② 時刻'''  
system_prompt = ""  
あなたはアシスタントです。  
「今何時？」など、現在時刻・日付を答える場合は必ず appRAG() を呼び出してください。  
"""
```

7. RAG 検索関数（appRAG）と補助ロジック-a)

```
'''② 時刻'''  
def appRAG(location: str | None = None) -> str:  
    now = datetime.datetime.now(ZoneInfo("Asia/Tokyo"))  
    return now.strftime("%Y-%m-%d %H:%M:%S (%Z)")
```

7. RAG 検索関数（appRAG）と補助ロジック-b)

```
'''② 時刻'''  
class AppRagArgs(BaseModel):  
    location: str | None = None
```

Function Calling - ③ 天気

最初の import/pip

```
'''③ 天気'''  
'''pip install requests'''  
import requests
```

3. Function Calling 用ツール定義

```
'''③ 天気'''  
tool_app_rag = {  
    "type": "function",  
    "name": "appRAG",  
    "description": "指定都市の現在の天気を取得します。省略時は Tokyo を使いま  
す。",  
    "parameters": {  
        "type": "object",  
        "properties": {  
            "location": {  
                "type": "string",  
                "description": "都市名 (例: Tokyo) "  
            }  
        },  
        "required": []  
    }  
}
```

4. システムプロンプト

```
'''③ 天気'''  
system_prompt = ""  
「東京の天気は？」など天気を答える場合は必ず appRAG(location) を呼び出してくだ  
さい。  
'''
```

7. RAG 検索関数 (appRAG) と補助ロジック-a)

```
'''③ 天気'''  
def appRAG(location: str | None = None) -> str:  
    city = location or "Tokyo"  
    url = f"https://wttr.in/{city}?format=j1"  
    try:  
        data = requests.get(url, timeout=4).json()  
        cond = data["current_condition"][0]  
        print(f"{city}の気温は{cond['temp_C']}°C、  
{cond['weatherDesc'][0]['value']}です。")  
        return f"{city}の気温は{cond['temp_C']}°C、  
{cond['weatherDesc'][0]['value']}です。"  
    except Exception as e:  
        return f"{city} の天気取得に失敗しました: {e}"
```

7. RAG 検索関数 (appRAG) と補助ロジック-b)

```
'''③ 天気'''  
class AppRagArgs(BaseModel):  
    location: str | None = None
```

Function Calling - ④ 最新ニュース

最初の import/pip

```
'''④ 最新ニュース'''  
'''pip install feedparser'''  
import feedparser
```

3. Function Calling 用ツール定義

```
'''④ 最新ニュース'''  
tool_app_rag = {  
    "type": "function",  
    "name": "appRAG",  
    "description": "Google News 日本語版の最新3件の見出しを取得します。",  
    "parameters": {  
        "type": "object",  
        "properties": {},  
        "required": []  
    }  
}
```

4. システムプロンプト

```
'''④ 最新ニュース'''  
system_prompt = ""  
「最新ニュースは？」など、ニュース見出しを答える場合は必ず appRAG() を呼び出してください。  
'''
```

7. RAG 検索関数 (appRAG) と補助ロジック-a)

```
'''④ 最新ニュース'''  
def appRAG() -> str:  
    rss = "https://news.google.com/rss?hl=ja&gl=JP&ceid=JP:ja"  
    feed = feedparser.parse(rss)  
    if not feed.entries:  
        return "ニュースを取得できませんでした。"  
    top3 = feed.entries[:3]  
    return "\n".join(f"{i+1}. {e.title}" for i, e in enumerate(top3))
```

7. RAG 検索関数 (appRAG) と補助ロジック-b)

```
'''④ 最新ニュース'''  
class AppRagArgs(BaseModel):  
    pass # 引数なし
```